

How Well Do Cheap Phishing Detectors Generalize? A Leakage-Aware, Cross-Corpus and Cost-Aware Benchmark of Small Open LLMs

Jahid Ibna Sinha
ID 011221376
United International University
Dhaka, Bangladesh

Md. Jakaria Alam Saimon
ID 011221002
United International University
Dhaka, Bangladesh

Nowshin Anjum Faria
ID 011221372
United International University
Dhaka, Bangladesh

Abstract—Published phishing email detectors, from GPT-4 prompts to fine-tuned transformers, report 97 to 99% accuracy. Almost all of these numbers come from training and testing inside one dataset. We ask how good cheap detectors are when the test mail comes from a corpus the model has never seen, and when emails shared between corpora are removed first. Comparing nine public email sources, we find that the widely used Kaggle phishing set contains 86% of SpamAssassin, 90% of Ling-Spam and 35% of Enron as near duplicates, while an exact-match check finds only 2 of the SpamAssassin copies. Removing the copies lowers the F1 of a TF-IDF model transferred from Kaggle to SpamAssassin from 0.966 to 0.666, and lowers leave-one-corpus-out F1 by up to 11.5 points. We then compare classical models, DistilBERT and six small open LLMs (1B to 14B parameters) on the same held-out emails, including AI-written phishing, next to the measured API cost. Zero-shot Qwen-2.5-7B gives the best balance, with a mean F1 of 0.928 on unseen corpora and 0.819 on AI-written phishing for 3 US cents per 1,000 emails. Few-shot examples taken from old corpora raise scores on old mail but lower them on AI-written phishing. The whole study ran on one laptop for 0.50 US dollars of API fees.

Index Terms—phishing detection, large language models, dataset leakage, near-duplicate detection, cross-corpus evaluation, cost

I. Introduction

Phishing email is still one of the most common ways into an organization. The attacker does not break any technical control. They write a message that looks like it came from a bank, a colleague or a delivery company, and the reader hands over the password. Generative AI made this cheaper. Heiding et al. [1] showed that phishing written by GPT-4 already gets click rates close to phishing written by human experts.

The defense side has moved to language models too, and the published numbers look excellent. ChatSpamDetector reports 99.70% accuracy with GPT-4 [2]. A fine-tuned RoBERTa reaches 98.45% [3]. A five agent LLM system reaches 97.89% [7]. When we reproduced the setting with a plain TF-IDF and Logistic Regression model, we also got 97 to 99% accuracy. A method older than deep learning matching GPT-4 is a warning sign. It suggests the benchmark, not the model, is doing the work.

Two things make these numbers hard to trust. First, most studies train and test inside a single corpus, while a real mail server sees mail that looks nothing like any training set. Second, the public corpora are not independent. Popular datasets such as the Kaggle phishing email set are aggregates built from older collections, so the same email can sit on both sides of a train and test split, even when the two sides come from “different” datasets. A third, practical point is cost. For a university or a small company, the price of running a detector matters as much as its accuracy, yet none of the LLM papers we reviewed report it.

This paper asks a simple question. If we remove the leaked emails and always test on a corpus the model has never seen, how good are cheap detectors, and what do they cost? Our contributions are:

- 1) A measurement of email overlap between nine public phishing and spam sources at three levels of strictness. We show that exact matching, the check most people would run, misses almost all of it.
- 2) A leave-one-corpus-out benchmark over six corpora in which every training email that has a near duplicate in the test corpus is removed, and a measurement of how much leaked emails inflate the score.
- 3) A comparison of classical models, a fine-tuned DistilBERT and six small open LLMs (1B to 14B parameters) on the same emails, including AI-written phishing, with the measured dollar cost of every LLM call.

The whole study cost under one US dollar in API fees and ran on one 8 GB laptop. Code, results and a notebook are public.¹

II. Related Work

LLM detectors. Koide et al. [2] prompt GPT-4 with a chain of thought and parse the verdict. Heiding et al. [1] test GPT-4, Claude, Bard and LLaMA as judges of suspicious mail. Lin et al. [4] show that 3B models

¹<https://github.com/Jahid15/phishing-llm-cross-corpus>

perform poorly out of the box but improve with prompt engineering, LoRA fine-tuning on explanations from a paid GPT-4o-mini teacher, and ensembling. They list the missing cost analysis as a limitation. MultiPhishGuard [7] combines five cooperating agents. Phishsense-1B [8] is a LoRA-tuned 1B model that reports 97.5% accuracy on its own data and 70% on a separate real-world set, the clearest sign in the literature that in-corpus scores do not transfer.

Encoders and explanations. Uddin et al. [3] fine-tune RoBERTa on the Kaggle set and add LIME based explanations. Kuikel et al. [5] find that the most accurate models are not the ones with the most faithful explanations. Alhuzali et al. [6] compare many classical and deep models on several public datasets.

Cross-dataset testing. The closest work to ours is E-PhishGen [9]. The authors reassess eight legacy corpora, show that a Logistic Regression model trained on CEAS falls from 0.98 F1 to 0.27 on average on other corpora, and release E-PhishLLM, a set of emails generated with GPT-4o-mini. We use E-PhishLLM as our AI-phishing test. Our work differs in three ways: we measure and remove near-duplicate leakage between corpora before testing, we include the Kaggle aggregate that much of the LLM literature uses, and we report measured cost next to accuracy.

Leakage in ML. Arp et al. [11] list data snooping and sampling bias among the most common pitfalls of machine learning in security. Kapoor and Narayanan [12] trace many published results in other fields to leakage between training and test data. Our overlap study is a concrete case of this problem in phishing detection.

III. Data

We use nine sources (Table I). Six contain both classes and serve for training and testing: SpamAssassin, CEAS-08, TREC-07, Ling-Spam and Enron from the curated collection of Champa et al. [10], and the Kaggle phishing email set used by [3]. Three are test-only. Nazario contains real credential phishing and Nigerian contains advance-fee fraud, both with no legitimate class. E-PhishLLM [9] contains phishing and legitimate emails written by GPT-4o-mini. We keep its English part.

For every source the model input is the subject and the body joined by a new line. Label 1 means phishing or spam and label 0 means legitimate. We drop bodies shorter than 20 characters and exact duplicates inside a corpus. Corpora larger than 10,000 emails are reduced to a stratified sample of 10,000 (seed 42) so that the three largest ones do not dominate training. From each source we also draw a fixed evaluation subset of 300 emails, balanced where both classes exist (150 for the one-class sets). All models are scored on these same emails, which makes the LLM and non-LLM numbers directly comparable.

TABLE I
Email sources after cleaning. “Used” is after the 10k stratified cap. “Eval” is the fixed subset every model is scored on.

Source	Role	Cleaned	Used	Positive	Eval
SpamAssassin	train/test	5,803	5,803	1,712	300
CEAS-08	train/test	39,091	10,000	5,574	300
TREC-07	train/test	53,638	10,000	5,460	300
Ling-Spam	train/test	2,858	2,858	458	300
Enron	train/test	29,604	10,000	4,689	300
Kaggle	train/test	17,505	10,000	3,731	300
Nazario	test	1,554	1,554	1,554	150
Nigerian	test	3,316	3,316	3,316	150
E-PhishLLM (en)	test	11,502	10,000	5,213	300

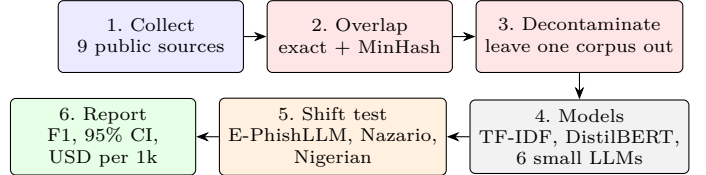


Fig. 1. The six steps of our pipeline. Steps 2 and 3 are what most prior work skips.

The positive class is not the same across corpora. SpamAssassin, CEAS, TREC and Ling mark ordinary spam as positive, while Nazario is pure phishing. We keep the labels as published and phrase the LLM prompt to match (“phishing, scam or spam”), and we report recall on Nazario and Nigerian separately.

IV. Method

Fig. 1 shows the pipeline. It has six steps, one script each.

A. Overlap between corpora

We compare every pair of the nine sources on the full cleaned corpora, before sampling, at three levels.

- Exact raw: MD5 of the text exactly as the model reads it.
- Exact normalized: MD5 of the body after lowercasing and removing all whitespace. This is the check we used in our preliminary work.
- Near duplicate: MinHash [13] with 128 permutations over 9-character shingles of the first 2,000 letters and digits of the body, with locality sensitive hashing to find candidates. Two emails count as the same if their estimated Jaccard similarity is at least 0.8.

We first tried 5-word shingles. They missed many copies, because the Kaggle aggregate often deletes line breaks and glues two words together (“cream?Isn’t”). Character shingles over letters and digits ignore that. To check the matcher, we took 400 random SpamAssassin emails that had no normalized exact match in Kaggle and searched Kaggle by brute force. 59% had a Kaggle email with a true Jaccard of at least 0.8, the same rate MinHash reported for that group, and manual reading confirmed they were

the same messages with different line breaks or a cut first word.

B. Leave-one-corpus-out with decontamination

For each of the six corpora T we train on the other five and test on all of T . We run this twice. In the raw setting the training data is used as it is. In the clean setting we first remove every training email that has a near duplicate in T . The difference in F1 between the two is the inflation caused by leaked emails. The three test-only sets are scored with a model trained on all six corpora, again with near duplicates of the test set removed. As a reference we also report the usual in-corpus score, an 80/20 stratified split inside each corpus.

C. Models

Classical. TF-IDF over word unigrams and bigrams (50,000 features, minimum document frequency 2) with Logistic Regression or multinomial Naive Bayes, as in our preliminary work.

DistilBERT. distilbert-base-uncased [14] fine-tuned for one epoch, 256 tokens, batch 16, learning rate 5×10^{-5} , on a stratified sample of 1,000 emails per training corpus (about 5,000 per fold). It runs on the laptop’s Apple M1 GPU. Only the clean setting is run.

Small open LLMs. Llama-3.2-1B, Llama-3.2-3B, Llama-3.1-8B, Qwen-2.5-7B, Gemma-3-12B and Phi-4 (14B), all instruction-tuned and called through OpenRouter. The system prompt asks whether the email is “malicious or unwanted (phishing, scam or spam) or legitimate” and to answer with one word. Temperature is 0, output is limited to 5 tokens, and emails are cut to 1,500 characters. An answer containing “phish”, “spam” or “malicious” counts as positive, one containing “legit” as negative. Anything else (for example a refusal) counts as negative and is reported. An LLM never sees our training data, so every test set is unseen for it. In the few-shot setting the prompt holds four short examples (two positive, two negative) drawn from corpora other than the test corpus, which keeps the leave-one-corpus-out rule.

D. Metrics and cost

We report F1 for the positive class, because accuracy hides the failure we care about (a model that calls everything legitimate still gets high accuracy on a mostly legitimate corpus). The headline robustness number is the mean F1 over the six held-out corpora, with a 95% interval from 1,000 bootstrap resamples of the emails in each test set, plus the worst of the six. For LLMs, cost is the sum of the usage.cost field that OpenRouter returns for every call, divided by the number of emails. Local models have no API cost. We report their inference time instead.

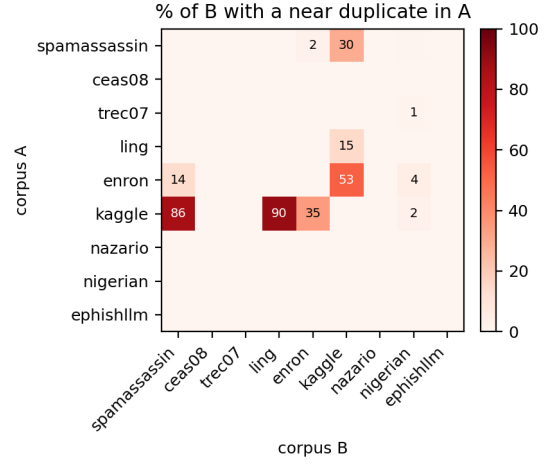


Fig. 2. Near-duplicate overlap. Cell (A, B) is the share of corpus B that has a near duplicate in corpus A .

TABLE II
Emails of one corpus found inside another, at three levels of matching. Percentages are of the first corpus.

Pair	Exact raw	No whitespace	Near duplicate
SpamAssassin in Kaggle	2 (<0.1%)	3,850 (66.3%)	5,009 (86.3%)
Ling-Spam in Kaggle	61 (2.1%)	62 (2.2%)	2,577 (90.2%)
Enron in Kaggle	3 (<0.1%)	187 (0.6%)	10,229 (34.5%)
Kaggle in Enron	3 (<0.1%)	188 (1.1%)	9,212 (52.6%)
SpamAssassin in Enron	0 (0.0%)	122 (2.1%)	793 (13.7%)
Nigerian in TREC-07	2 (<0.1%)	2 (<0.1%)	42 (1.3%)

V. Results

A. The corpora overlap far more than exact matching shows

Table II and Fig. 2 show the overlap. Compared exactly as a model reads them, SpamAssassin and Kaggle share only 2 emails. With whitespace removed they share 3,850, and with near-duplicate matching 5,009, which is 86% of SpamAssassin. The Kaggle set also contains 90% of Ling-Spam and 35% of Enron, and more than half of Kaggle has a copy in Enron. In other words, the Kaggle “dataset” is mostly three older corpora put together, with formatting changed on the way. Anyone who deduplicates the usual way, by exact text, would call these corpora independent. The older corpora also overlap each other: 14% of SpamAssassin has a near copy in Enron.

CEAS-08, TREC-07, Nazario and E-PhishLLM share almost nothing with any other source. This gives us a natural control group for the next experiment.

B. Leaked emails inflate cross-corpus scores

Table III repeats the one-corpus-to-one-corpus test from our preliminary work. The Kaggle to SpamAssassin transfer, which looked almost perfect (F1 0.966), falls to 0.666 once the 2,935 Kaggle training emails that also appear in SpamAssassin are removed. Kaggle to Enron

TABLE III

Single-corpus transfer with TF-IDF + LogReg, before and after removing training emails that also appear in the test corpus.

Train → test	Removed	Raw F1	Clean F1
Kaggle → SpamAssassin	2,935	0.966	0.666
Kaggle → Enron	5,294	0.969	0.738
Kaggle → Ling-Spam	1,436	0.979	0.840
SpamAssassin → Kaggle	5,009	0.766	0.596
SpamAssassin → Enron	793	0.673	0.473

TABLE IV

Leave-one-corpus-out F1 of TF-IDF + Logistic Regression on the full held-out corpus. “Removed” is the number of training emails with a near duplicate in the test corpus. Δ is raw minus clean, in F1 points, for LogReg and Naive Bayes.

Held out	Removed	In-corpus	Raw	Clean	Δ LR	Δ NB
SpamAssassin	3,165	0.966	0.947	0.847	+10.0	+5.3
Ling-Spam	1,444	0.930	0.962	0.847	+11.5	+4.5
Enron	6,091	0.981	0.972	0.919	+5.4	+5.4
Kaggle	11,018	0.965	0.976	0.928	+4.8	+2.1
CEAS-08	13	0.986	0.833	0.832	+0.1	-0.0
TREC-07	7	0.985	0.871	0.871	+0.0	+0.1

falls from 0.969 to 0.738. The “good transfer” was the model recognising emails it had already seen.

Table IV shows the same effect in the leave-one-corpus-out setting. Removing leaked emails lowers Logistic Regression F1 by 11.5 points on Ling-Spam, 10.0 on SpamAssassin, 5.4 on Enron and 4.8 on Kaggle. For CEAS-08 and TREC-07, which share almost nothing with the other corpora, the change is 0.1 points or less. The inflation appears exactly where the overlap is, which is strong evidence that leakage, and not some other difference between the settings, causes it.

Fig. 3 puts the three settings side by side. The usual in-corpus split gives 0.93 to 0.99. Honest leave-one-corpus-out testing gives 0.83 to 0.93. The in-corpus number overstates what the model will do on mail from a new source by 4 to 15 points.

C. Comparing all models on unseen mail

Table V is the table our proposal promised: every model on the same held-out emails, with robustness, AI phishing and cost in one place. Fig. 4 breaks the scores down per test set.

Small LLMs can match or beat trained models on unseen corpora. Zero-shot Qwen-2.5-7B reaches a mean F1 of 0.928 on the six held-out corpora, above TF-IDF + LogReg (0.901) and although Qwen never saw any of our training data.

AI-written phishing separates the models. On E-PhishLLM the trained models fall to 0.640 (LogReg) and 0.631 (Naive Bayes). They learned what old spam looks like, and polished GPT-4o-mini phishing does not look like old spam. Qwen-2.5-7B (0.819) and Llama-3.1-8B (0.817) hold up much better.

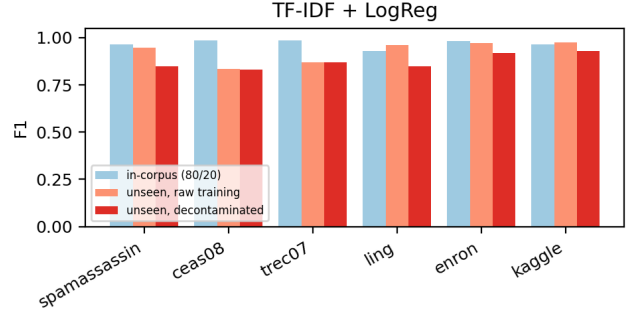


Fig. 3. TF-IDF + LogReg F1 per held-out corpus: in-corpus split, unseen corpus with raw training data, and unseen corpus after decontamination.

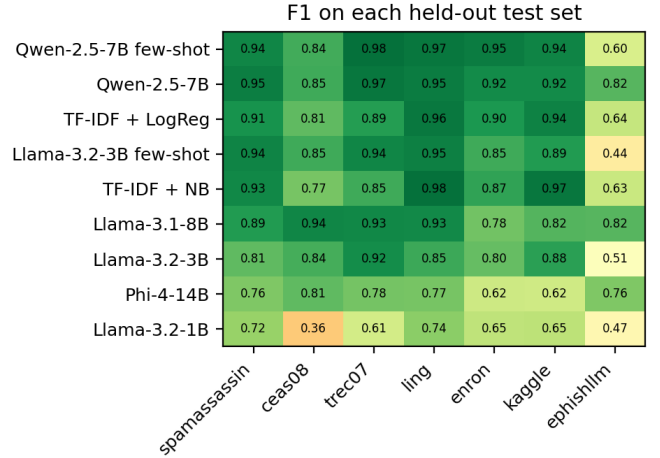


Fig. 4. F1 of every model on each held-out test set.

Few-shot examples help on old mail and hurt on new mail. Four examples drawn from the other corpora raise Qwen-2.5-7B from 0.928 to 0.937 on the held-out corpora, and Llama-3.2-3B from 0.850 to 0.901. The same examples lower AI-phishing F1 from 0.819 to 0.598 for Qwen and from 0.507 to 0.438 for Llama. The examples come from old corpora, and they seem to anchor the model to what phishing looked like in 2002 to 2008. Few-shot also more than doubles the cost, because the prompt is longer.

Bigger is not automatically better. Phi-4 (14B) has a lower mean F1 (0.727) than the 3B and 8B Llama models. In 37% of emails it ignored the instruction to answer in one word and started an explanation (“As a large language model...”, “Based on the content...”), so no verdict fit in the output. Llama-3.2-1B is too weak for the task (0.623, with a worst-corpus F1 of 0.359).

D. Cost

The measured cost of zero-shot classification ranges from 0.009 to 0.031 US dollars per 1,000 emails (Fig. 5). Qwen-2.5-7B costs 0.031 dollars per 1,000 emails, so screening 100,000 emails a month would cost about 3.07

TABLE V

Main results. All numbers are on the same fixed evaluation emails. Trained models use decontaminated leave-one-corpus-out data. “Unseen” is the mean F1 over the six held-out corpora with a 95% bootstrap interval, “Worst” the lowest of the six. AI phishing is F1 on E-PhishLLM. Nazario and Nigerian contain only positives, so we report recall. Cost is the measured OpenRouter charge in USD per 1,000 emails.

Model	In-corpus	Unseen	95% CI	Worst	AI phishing	Nazario rec.	Nigerian rec.	USD/1k
Qwen-2.5-7B few-shot	—	0.937	0.925–0.949	0.844	0.598	0.847	1.000	0.074
Qwen-2.5-7B	—	0.928	0.914–0.940	0.850	0.819	0.940	1.000	0.031
TF-IDF + LogReg	0.969	0.901	0.886–0.916	0.807	0.640	0.713	1.000	local
Llama-3.2-3B few-shot	—	0.901	0.886–0.916	0.848	0.438	0.680	0.993	0.037
TF-IDF + NB	0.929	0.895	0.879–0.909	0.773	0.631	0.600	0.987	local
Llama-3.1-8B	—	0.881	0.866–0.896	0.779	0.817	0.973	1.000	0.011
Llama-3.2-3B	—	0.850	0.832–0.869	0.799	0.507	0.827	1.000	0.016
Phi-4-14B	—	0.727	0.703–0.749	0.621	0.759	0.660	0.513	0.021
Llama-3.2-1B	—	0.623	0.599–0.647	0.359	0.474	0.760	0.607	0.009

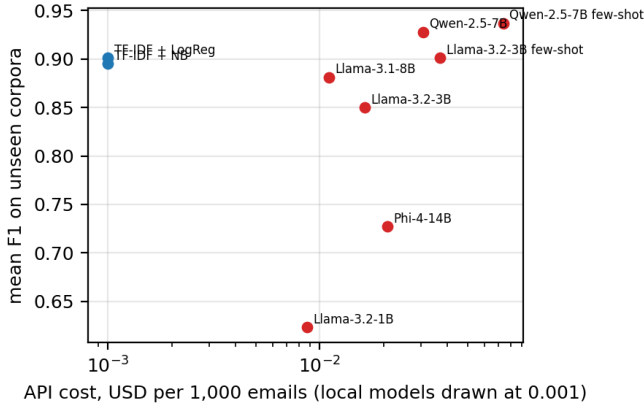


Fig. 5. Mean F1 on unseen corpora against measured API cost. Local models have no API cost and are drawn at 0.001.

dollars. TF-IDF needs about 0.25 CPU seconds per 1,000 emails and no API. DistilBERT needs 0 minutes of laptop GPU time per training fold and 0 seconds per 1,000 emails. The whole study, 18,000 LLM calls including discarded trials, cost 0.50 US dollars.

VI. Discussion

Check overlap before claiming generalization. Our clearest finding is about the benchmarks, not the models. If a paper trains on the Kaggle set and tests on SpamAssassin, most of the test set is already in the training data. Exact-match deduplication does not catch it, because the aggregate changed the line breaks. A near-duplicate pass takes about five minutes on a laptop for 165,000 emails, and it should come before any cross-dataset claim.

In-corpus scores are not deployment scores. The 97 to 99% in the literature is real, but it answers the question “how well does the model recognise more mail from the same collection”. A mail server does not get mail from the training collection. On unseen corpora the same TF-IDF model drops by 4 to 15 points, and on AI-written phishing by about 30.

What we would deploy. For an organisation with no GPU and a small budget, zero-shot Qwen-2.5-7B is the best balance we found. It is the most robust model across old corpora and one of the two best on AI-written phishing, at about three cents per thousand emails. TF-IDF remains a useful, free first filter for bulk spam, but it should not be trusted on modern phishing. Few-shot prompting needs care: examples must look like the mail the system will actually see, or they make it worse.

Output format is part of the model’s quality. Phi-4’s score is low partly because it did not follow the one-word format. In a real pipeline an answer that cannot be parsed is a missed detection, so format following matters as much as reasoning ability.

VII. Limitations

- Labels. Four of the six training corpora count ordinary spam as positive. We keep the published labels and report phishing-only recall on Nazario separately, but a clean phishing versus spam relabelling is still missing.
- Sample size. Each test set has 300 emails (150 for the one-class sets), which keeps cost low. The 95% intervals of the mean F1 are about ± 0.02 , so differences below that are not meaningful.
- LLM pre-training. The old public corpora may be part of the LLMs’ pre-training data. E-PhishLLM was released in 2025, after most of these models, which makes it the most trustworthy test for them.
- Scope. Text only, English only, one prompt, one run per model. Headers, URLs and attachments are ignored. OpenRouter providers can change the model behind a name without notice.
- DistilBERT is trained lightly (1,000 emails per corpus, one epoch) to fit an 8 GB laptop, so it is a lower bound for fine-tuned encoders.
- Phishsense-1B, which we planned to include, needs a HuggingFace access token and was not run.

VIII. Conclusion

Public phishing corpora overlap heavily, mostly in ways that exact matching cannot see. The Kaggle set contains

most of SpamAssassin and Ling-Spam and a third of Enron. Removing these copies lowers cross-corpus F1 by up to 30 points for a single pair and up to 11.5 points in leave-one-corpus-out testing. On honest, unseen test data, a small open LLM used zero-shot is as robust as trained models on old mail and much better on AI-written phishing, for a few cents per thousand emails. High accuracy on one dataset is easy. Honest evaluation is the hard part. All code, results and a Colab notebook are public so that others can check and extend this work.

Acknowledgment

We thank our Computer Security course instructor at United International University for the feedback on our literature review and proposal.

References

- [1] F. Heiding, B. Schneier, A. Vishwanath, J. Bernstein, and P. S. Park, “Devising and detecting phishing emails using large language models,” *IEEE Access*, vol. 12, pp. 42131–42146, 2024.
- [2] T. Koide, N. Fukushi, H. Nakano, and D. Chiba, “ChatSpamDetector: Leveraging large language models for effective phishing email detection,” *arXiv:2402.18093*, 2024.
- [3] M. A. Uddin, M. Mahiuddin, and I. H. Sarker, “An explainable transformer-based model for phishing email detection: A large language model approach,” *arXiv:2402.13871*, 2024.
- [4] Z. Lin, Z. Liu, and H. Fan, “Improving phishing email detection performance of small large language models,” *arXiv:2505.00034*, 2025.
- [5] S. Kuikel, A. Piplai, and P. Aggarwal, “Evaluating large language models for phishing detection, self-consistency, faithfulness, and explainability,” *arXiv:2506.13746*, 2025.
- [6] A. Alhuzali, A. Alloqmani, M. Aljabri, and F. Alharbi, “In-depth analysis of phishing email detection: Evaluating the performance of machine learning and deep learning models across multiple datasets,” *Applied Sciences*, vol. 15, no. 6, 2025.
- [7] Y. Xue, E. Spero, M. W. Woo, W. Gao, and G. Russello, “MultiPhishGuard: An explainable and adaptive multi-agent LLM system for phishing email detection,” *arXiv:2505.23803*, 2025.
- [8] S. E. Blake, “Phishsense-1B: A technical perspective on an AI-powered phishing detection model,” *arXiv:2503.10944*, 2025.
- [9] L. Pajola, E. Caripoti, S. Banzer, S. Pizzi, M. Conti, and G. Apruzzese, “E-PhishGen: Unlocking novel research in phishing email detection,” in *Proc. ACM Workshop on Artificial Intelligence and Security (AISec)*, 2025.
- [10] A. I. Champa, M. F. Rabbi, and M. F. Zibran, “Curated datasets and feature analysis for phishing email detection with machine learning,” in *Proc. IEEE 3rd Int. Conf. Computing and Machine Intelligence (ICMI)*, 2024, pp. 1–7.
- [11] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, “Dos and don’ts of machine learning in computer security,” in *Proc. USENIX Security Symposium*, 2022, pp. 3971–3988.
- [12] S. Kapoor and A. Narayanan, “Leakage and the reproducibility crisis in machine-learning-based science,” *Patterns*, vol. 4, no. 9, 2023.
- [13] A. Z. Broder, “On the resemblance and containment of documents,” in *Proc. Compression and Complexity of Sequences*, 1997, pp. 21–29.
- [14] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter,” *arXiv:1910.01108*, 2019.